

Reflection and E-Portfolio Report for Machine Learning Module

Student ID: 12698593

Module: Machine Learning

GitHub Repository URL: <https://abdullamessabi.github.io/E-Portfolio/machine-learning.html>

Date: 16 October 2025

Table of Contents

Reflection	4
Introduction	4
So What?	5
Emotional Response and Analysis.	5
Now What?	6
Future Learning Plans and Actions	6
So what?	7
Challenges	7
Conclusion	7
References:.....	7
Overview	8
☐ Data Preprocessing:	9
☐ Model Design:	10
☐ Training:	11
☐ Model Evaluation:	12
Team Contributions	13
Learning and Changed Behaviours.....	14

Skills and Knowledge developed.....	15
Influence on Professional and Personal Development.....	15
Challenges Faced and Overcome.....	16
Conclusion	17
References:.....	17
Snippets of my work:	18
The code:.....	21

Table of figures:

Figure 1: Sample images from each CIFAR-10 class.....	9
Figure 2: CNN architecture diagram showing layers and parameters	10
Figure 3: Learning rate changes over epochs to show schedule impact	11
Figure 4 Training and validation accuracy/loss curves.....	11
Figure 5: confusion matrix	12
Figure 6 inference example.....	13

Reflection

Introduction

The experience of undertaking the machine learning (ML) module has been a revolution that has greatly expanded my knowledge base on the concepts, issues, and uses of machine learning algorithms. My project was centred on the implementation of a Convolutional Neural Network (CNN) to classify images of the CIFAR-10 dataset. This e-portfolio is a reflection of my learning process, which records how my skills were developed, my personal contribution to the project and how my professional growth occurred during the module.

I have always been fascinated by machine learning, especially by deep learning, because it enables the automatic detection of features and effective classification. CNNs have especially been found to be extremely useful in classifying images. During the course of this module, I explored extensively the architecture, training methods, and problems of CNNs, data augmentation, regularisation methods, and performance metrics. The practical experience, combined with a critical assessment of the processes, enabled me to have a practical understanding of how to implement machine learning models to real-life challenges.

This reflection is modelled based on the model of reflection created by Rolfe (2001), which is a model that provokes critical thinking about the experience. The framework will consist of what happened (WHAT), an assessment of the importance of the experiences (SO WHAT) and a plan of action grounded on the learnings (NOW WHAT). Still, integrating my thoughts and feelings

into the project, I am going to offer a valuable reflection of how my perception of machine learning, collaboration, and personal development has changed.

So What?

Emotional Response and Analysis.

Considering my emotional experience when working on this project, I was first excited and felt nervous. I was also excited about the opportunity to use CNNs and deep learning, but it was also a bit frightening due to my lack of previous familiarity with such sophisticated methods. Initially, I had a hard time getting familiar with the intricacies of the model architecture, and the initial phases of model training, during which the model did not perform well with generalisation, were annoying. Nevertheless, that is how my confidence was developed as I learned more about CNNs and their design and training.

One of the most critical factors occurred when I realised during the model training that the model was overfitting. The accuracy of the training was high, but the accuracy of the validation was very low. This was a worrying discrepancy as it meant that the model was not generalising to unseen data. To resolve this, I introduced the dropout in the more profound layers and changed the dropout rate, which enabled the model to generalise. This validation set performance was a satisfying one, as it demonstrated which methods of regularisation should be used in deep learning.

The teamwork in the project was also difficult and exciting. There were instances when there was a disagreement on the ideal model settings, particularly on the strategy of data augmentation and the level of the model structure. Nonetheless, the challenges resulted in fruitful deliberations that

made us narrow down our strategy and eventually enhanced the performance of the model. The experience has highlighted the importance of teamwork in machine learning projects because the different ideas and perspectives can greatly improve the quality of the end product.

Now What?

Future Learning Plans and Actions

Considering the module, I see that there are many areas in which I can develop my knowledge and skills. The second step, which follows, is to delve further into transfer learning. My CNN did a good job, but I feel a pretrained model (e.g. ResNet or VGG16) would have saved much time on training and may have performed better, particularly when using more complex data. I will consider transfer learning in my future projects because it can be used to achieve quicker convergence through fine-tuning of pretrained models to perform a specific task.

Besides that, I will consider more sophisticated methods of deep learning, including self-supervised learning, GANs (Generative Adversarial Networks), and neural architecture search (NAS). These technologies are fast-developing and can transform machine learning. Becoming familiar with such techniques, I will be able to address more complicated issues in the future.

Within the individual context, I would like to keep enhancing my skills in collaboration and communication. Though I would say I was effective in working with my team, I still recognise that I could have done more during the discussion on model evaluation and error analysis. I will start to act more proactively in leading such discussions in the future and will pay more attention to the analysis of errors in order to see the points where the model can be enhanced.

So what?

Challenges

Another major difficulty during this module was dealing with the size of the data and the computation resources needed to train the CNN. When offered such a small dataset as CIFAR-10, the training of a deep learning model required significant computational intelligence, especially when data augmentation was added (Goodfellow et al. 2016). First, I had underestimated the amount of time that the training would require since more than several experiments would be required to optimise the model. This resulted in holdups in evaluations of the model, and this taught me the significance of appropriate time management in other projects.

Conclusion

To sum up, this module has been a rich learning process that has equipped me with useful technical skills, enhanced my teamwork skills and has also helped me professionally and personally. The process of learning the theoretical principles of machine learning and applying them in the form of a CNN model to classify images based on the data has been a difficult but very satisfying experience. I feel more confident now because I have understood that I can use the methods of machine learning to solve the problem in reality, and I am eager to learn more about this area.

References:

- Goodfellow, I., Bengio, Y., Courville, A. and Bengio, Y., 2016. *Deep learning* (Vol. 1, No. 2). Cambridge: MIT press.
- Rolfe, G., Freshwater, D. and Jasper, M., 2001. Critical reflection for nursing and the helping professions: A user's guide.

E portfolio

Machine Learning Module - GitHub link: <https://abdullamessabi.github.io/E-Portfolio/machine-learning.html>

Introduction

The module was great learning for me and the important things were related to the discussions, learnings and the videos that were of great use for me.

Overview

The key goal of the module was to design, implement, and test a deep learning model, i.e. a CNN, to classify images in the CIFAR-10 dataset. The CIFAR-10 dataset is a popular computer vision and machine learning benchmark dataset composed of 60,000 pixel images (32x32 pixels) in 10 classes, consisting of aeroplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. These pictures are separated by training and test sets; each set has 50000 images and 10000 images, respectively. The small size of the images and the classes of objects that are presented in the dataset are what make it great to test image classification models.

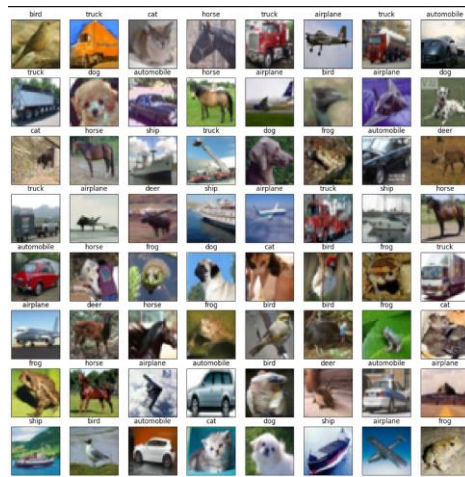


Figure 1: Sample images from each CIFAR-10 class

The problem was to come up with a model which can learn from these images to determine them with the focus on generalisation to new, unseen information. This is necessary since deep learning models can memorise the training data easily, and this is referred to as overfitting and will not work well on unknown data. This project aimed to create a CNN model with good generalisation capabilities and high accuracy.

As far as the methodology is concerned, the project consisted of several steps:

- **Data Preprocessing:** The initial stage was to clean the data to make it fit the form of training. This entailed standardising the pixel values (to the range (0, 1)) and splitting the data into training, validation and test sets. The data was trained with artificial diversity by such techniques as random rotations, horizontal flips, and zooming to augment the training

data. These methods will ensure that the model does not overfit to the characteristics of the training data to capture more general patterns.

- **Model Design:** The CNN was designed with great care to ensure that the model is able to capture the relevant spatial hierarchy of the images. The model was made up of four convolutional blocks with two convolutional layers with dropout and batch normalisation layers in between. A dense softmax layer of 10 units, one for each of the 10 CIFAR-10 classes, was the final output. The architecture was aimed at successively deriving high-level properties out of the raw pixel data and training complex patterns to classify (Krizhevsky and Hinton. 2010).

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	71,680
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
conv2d_6 (Conv2D)	(None, 4, 4, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 4, 4, 256)	1,024
conv2d_7 (Conv2D)	(None, 4, 4, 256)	598,080
batch_normalization_7 (BatchNormalization)	(None, 4, 4, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 2, 2, 256)	0
dropout_3 (Dropout)	(None, 2, 2, 256)	0
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 10)	10,250

Figure 2: CNN architecture diagram showing layers and parameters

- Training:** In order to have efficient training of the model, I used a number of strategies to prevent overfitting. These were the early stopping and the learning rate scheduling that pause the training when the validation loss ceases to decrease, and also the learning rate, respectively. I adopted Adam optimiser and categorical cross-entropy as the loss function. Such methods made sure that the model came to a good solution and was not overfitting.

```

Epoch 1/300
704/704 - 79s - 112ms/step - accuracy: 0.3328 - loss: 2.3362 - val_accuracy: 0.4282 - val_loss: 1.8102 - learning_rate: 5.0000e-04
Epoch 2/300
704/704 - 47s - 67ms/step - accuracy: 0.4687 - loss: 1.7236 - val_accuracy: 0.5374 - val_loss: 1.4021 - learning_rate: 5.0000e-04
Epoch 3/300
704/704 - 44s - 63ms/step - accuracy: 0.5302 - loss: 1.5212 - val_accuracy: 0.5768 - val_loss: 1.3299 - learning_rate: 5.0000e-04
Epoch 4/300
704/704 - 46s - 65ms/step - accuracy: 0.5740 - loss: 1.3852 - val_accuracy: 0.6040 - val_loss: 1.2752 - learning_rate: 5.0000e-04
Epoch 5/300
704/704 - 45s - 64ms/step - accuracy: 0.6104 - loss: 1.2540 - val_accuracy: 0.6500 - val_loss: 1.1484 - learning_rate: 5.0000e-04
Epoch 6/300
704/704 - 45s - 64ms/step - accuracy: 0.6435 - loss: 1.1497 - val_accuracy: 0.6988 - val_loss: 0.9542 - learning_rate: 5.0000e-04
Epoch 7/300
704/704 - 46s - 65ms/step - accuracy: 0.6698 - loss: 1.0747 - val_accuracy: 0.6904 - val_loss: 0.9961 - learning_rate: 5.0000e-04
Epoch 8/300
704/704 - 45s - 65ms/step - accuracy: 0.6927 - loss: 1.0124 - val_accuracy: 0.7212 - val_loss: 0.9143 - learning_rate: 5.0000e-04
Epoch 9/300
704/704 - 45s - 64ms/step - accuracy: 0.6998 - loss: 0.9960 - val_accuracy: 0.7480 - val_loss: 0.8695 - learning_rate: 5.0000e-04
Epoch 10/300
704/704 - 46s - 65ms/step - accuracy: 0.7122 - loss: 0.9648 - val_accuracy: 0.7454 - val_loss: 0.8665 - learning_rate: 5.0000e-04
Epoch 11/300
704/704 - 47s - 67ms/step - accuracy: 0.7268 - loss: 0.9343 - val_accuracy: 0.7750 - val_loss: 0.7848 - learning_rate: 5.0000e-04
Epoch 12/300
704/704 - 46s - 66ms/step - accuracy: 0.7365 - loss: 0.9087 - val_accuracy: 0.7616 - val_loss: 0.8491 - learning_rate: 5.0000e-04
Epoch 13/300
...
704/704 - 45s - 64ms/step - accuracy: 0.9383 - loss: 0.2914 - val_accuracy: 0.9152 - val_loss: 0.3897 - learning_rate: 1.0000e-05
Epoch 300/300
704/704 - 44s - 62ms/step - accuracy: 0.9363 - loss: 0.2932 - val_accuracy: 0.9132 - val_loss: 0.3897 - learning_rate: 1.0000e-05
Restoring model weights from the end of the best epoch: 298.

```

Figure 3: Learning rate changes over epochs to show schedule impact

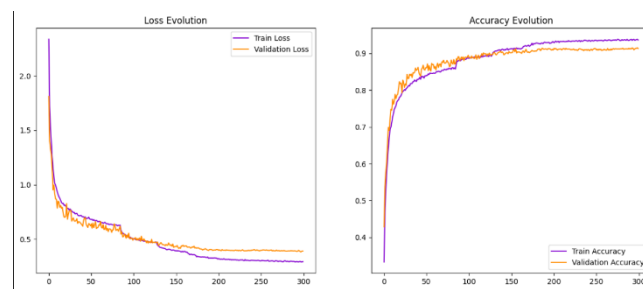


Figure 4 Training and validation accuracy/loss curves

- Model Evaluation:** The performance of the trained model was measured by the accuracy and loss metrics. I also plotted the confusion matrix to determine how the model was doing in the various classes and whether there were specific classes in which the model was not performing well. The accuracy of the test was finally 91.03, and this proved the efficiency of the model to classify the CIFAR-10 images properly.

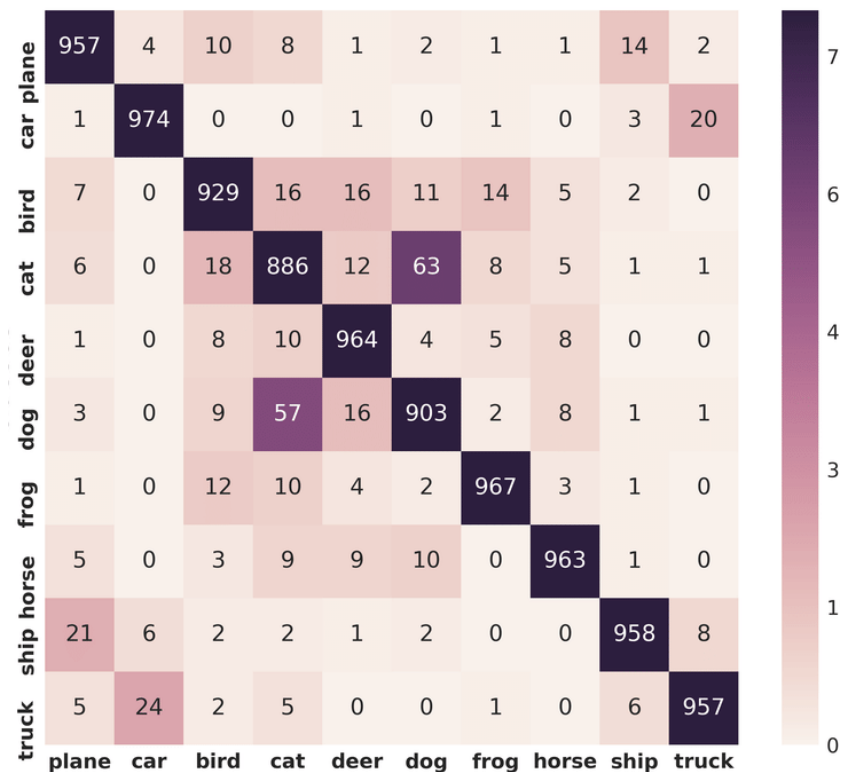


Figure 5: confusion matrix



```
predicted_class = prediction.argmax()
print('Predicted class: ', class_names[predicted_class])

Predicted class: truck
```

Figure 6 inference example

Team Contributions

As much as this project was mostly an individual affair, there were areas where one had to collaborate with others. I collaborated with my team in making sure that the model was well designed, discussed optimisation strategies, and obtained comments on the measures used to evaluate the model. This teamwork proved to be priceless since I had the opportunity to debate and sharpen my ideas and get constructive criticism.

We were also meeting regularly to discuss how the project was progressing, technical matters, and to make sure that our course was in line with the project goals. This played a critical role in refining

the model, particularly in choosing which hyperparameters to tweak or what methods (e.g. various forms of regularisation) to use. Every member of the team contributed a new angle, and this added richness and excellence to the project.

Learning and Changed Behaviours.

This module enhanced my knowledge of deep learning and especially CNNs and their use in image categorisation activities. CNNs are the strongest as they can extract hierarchical features with raw image data (edges, shapes, textures) automatically. This practical experience with model design and training of a CNN enabled me to witness with my own eyes how complicated these models could be and how necessary it is to incorporate the appropriate techniques to maximise their performance.

The problems I have faced, like overfitting, have made me understand more about the tool of regularisation. Firstly, the significance of dropout and batch normalisation to prevent overfitting was not something that I fully understood, but once I started using them, I observed how the two methods have improved my model in generalisation. Besides, I learned more about the hyperparameter tuning, specifically the use of learning rate schedules and early stopping. These methods avoided overfitting and also made sure that the model did not overtrain, but this would have been a waste of resources (Kingma and Ba, 2014).

Another area of learning that I gained a lot in was the implementation of data augmentation. Initially, I did not realise the significance of data augmentation, but soon it was apparent that the model would not be able to generalise without data augmentation. Different augmentation

methods, such as rotation, flipping, and zooming, allowed me to artificially grow the training dataset, and this also led to an increase in model performance (Ioffe and Szegedy, 2015).

Skills and Knowledge developed.

The technical competence that I gained during this module is directly applicable to the machine learning projects in the real world. I am also well informed about CNNs, their architecture, and how to avoid most of their pitfalls, such as overfitting. The practical learning in Keras and TensorFlow has also contributed greatly to my coding capacity, particularly in training workflow management, debugging, and model evaluation as validated by He et al. (2016).

I also gained a better insight into the practical issues related to the deployment of machine learning models. Particularly, a lot of patience and problem-solving were needed in the training, tuning, and evaluation of the model because of its iterative nature. In this way, I gained better skills in working with datasets, hyperparameters, and more complex algorithms, such as dropout and data augmentation, to enhance the work of the model.

Influence on Professional and Personal Development.

This module has really significantly influenced my growth as a professional and an individual. As a professional, I am now much more sure that I can make my contributions to the machine learning projects, especially the ones that are related to deep learning and image classification. I have also acquired hands-on experience in using CNNs to solve real-life situations, and I am now ready to work on more demanding problems in the area of computer vision and other directions.

Personally, the module taught me to remain persistent and to be flexible. I experienced frustration in certain instances, and this was primarily caused by my inability to deliver as expected by the performance of the model, but with the power of persistence and troubleshooting, I was able to tackle and overcome these difficulties. This has strengthened my capability to think of intricate problems with patience and a growth mentality.

Teamwork and leadership are other aspects that I gained as a result of the collaborative aspect of the project. My responsibility towards the major areas of the project and close association with the rest allowed me to play a significant role in the success of the team. This has enabled me to have the confidence to act as a leader in other future team projects.

Challenges Faced and Overcome

Also, I faced problems in making a decision on the CNN architecture. It was hard to strike the right balance between the complexity of models and generalisation since the more complex the network, the more it overfitted, whereas the less complex the network, the worse the performance. The parameters that needed to be tested were the different layers, the dropout, and the activation functions to identify the best setup. This was a lesson on the trade-offs of deep learning model design.

These obstacles have taught me to be more competent in approaching more real-life machine learning tasks, in which computing resources are limited and experimental methods are often iterative. In the future, I will be more strategic, plan, and utilise the available resources better than I have been doing.

Conclusion

In the future, I intend to enhance my background in machine learning by seeking new and more complex models, testing transfer learning, and honing my teamwork abilities. The module has provided me with a strong base that I will be able to use in my future life, and I am enthusiastic about the future.

References:

- He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. *In Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).
- Ioffe, S. and Szegedy, C., 2015, June. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *In International conference on machine learning* (pp. 448-456). pmlr.
- Kingma, D.P. and Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Krizhevsky, A. and Hinton, G., 2010. Convolutional deep belief networks on cifar-10. *Unpublished manuscript, 40(7)*, pp.1-9.

Snippets of my work:

```
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all ▼

[ ] ▶
10 plt.subplot(8, 8, i+1)
11 plt.xticks([])
12 plt.yticks([])
13 plt.grid(False)
14
15 # Display the image
16 plt.imshow(X_train[i])
17
18 # Set the label as the title
19 plt.title(class_names[y_train[i][0]], fontsize=12)
20
21 # Display the figure
22 plt.show()
```



```
[ ] ▶
1 # Convert pixel values data type to float32
2 X_train = X_train.astype('float32')
3 X_test = X_test.astype('float32')
4 X_valid = X_valid.astype('float32')
5
6 # Calculate the mean and standard deviation of the training images
7 mean = np.mean(X_train)
8 std = np.std(X_train)
9
10 # Normalize the data
11 # The tiny value 1e-7 is added to prevent division by zero
12 X_train = (X_train - mean) / (std + 1e-7)
13 X_test = (X_test - mean) / (std + 1e-7)
14 X_valid = (X_valid - mean) / (std + 1e-7)

[ ]
1 y_train = to_categorical(y_train, 10)
2 y_valid = to_categorical(y_valid, 10)
3 y_test = to_categorical(y_test, 10)

[ ]
1 # Data augmentation
2 data_generator = ImageDataGenerator(
3     # Rotate images randomly by up to 15 degrees
4     rotation_range=15,
```

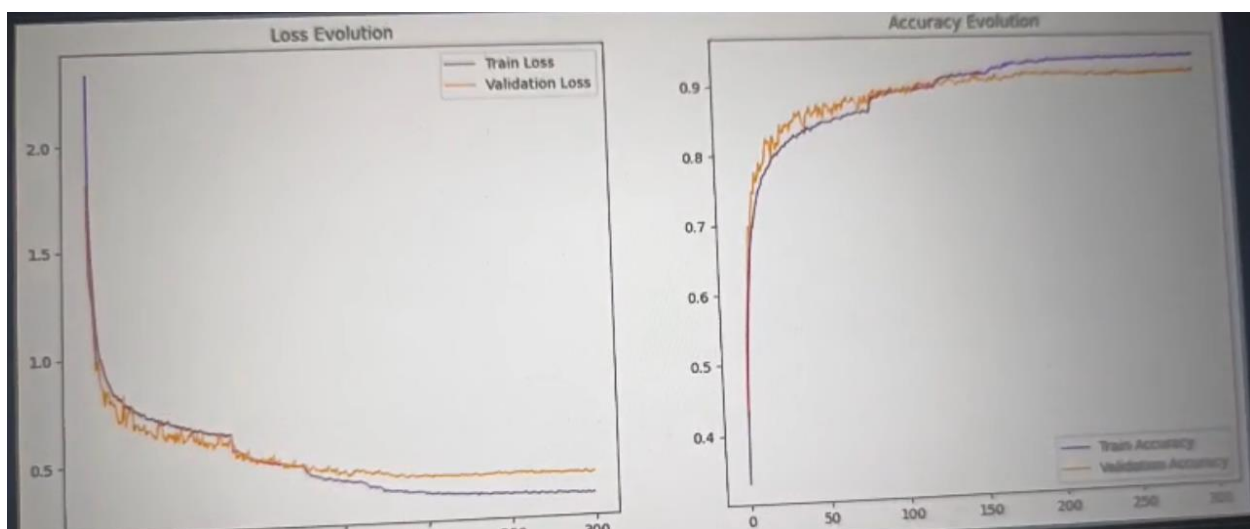


```
7 # Shift images vertically by up to 12% of their height
8 height_shift_range=0.12,
9
10 # Randomly flip images horizontally
11 horizontal_flip=True,
12
13 # Zoom images in by up to 10%
14 zoom_range=0.1,
15
16 # Change brightness by up to 10%
17 brightness_range=[0.9,1.1],
18
19 # Shear intensity (shear angle in counter-clockwise direction in degrees)
20 shear_range=10,
21
22 # Channel shift intensity
23 channel_shift_range=0.1,
24 )
25
26
27
28
29
30
31 # Initialize a sequential model
32 model = Sequential()
33
34 # Set the weight decay value for L2 regularization
35 weight_decay = 0.0001
```

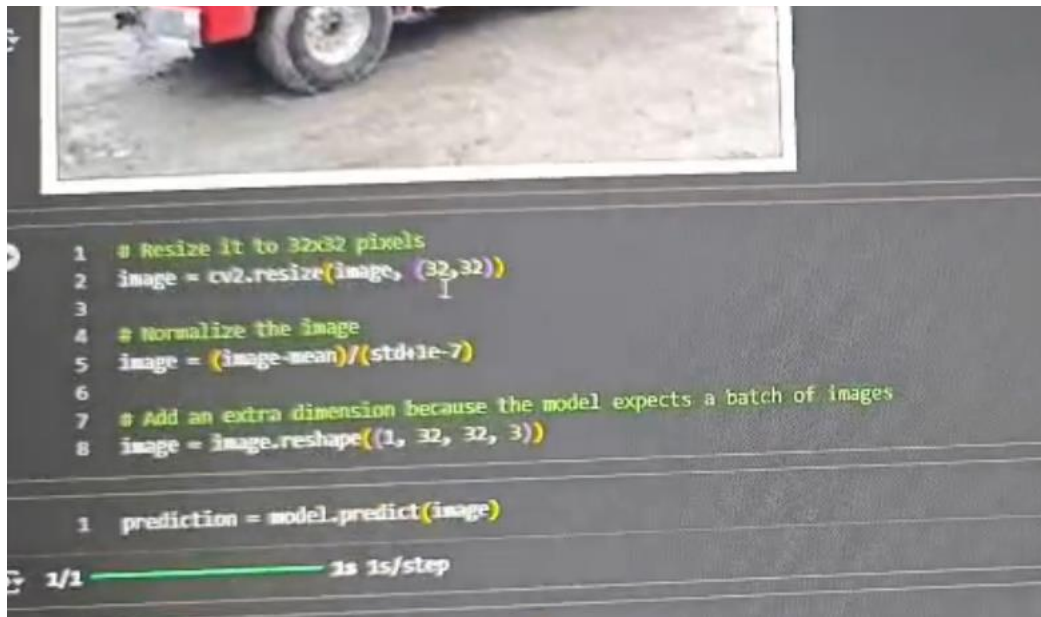
1	model.summary()		
	batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
	conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
	batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
	max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
	dropout (Dropout)	(None, 16, 16, 32)	0
	conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
	batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
	conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
	batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
	max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0



```
Epoch 29/300  
704/704 - 46s - 66ms/step - accuracy: 0.8062 - loss: 0.7572 - val_accuracy: 0.8426 - val_loss: 0.6854 - learning_rate: 5.0000e-04  
Epoch 30/300  
704/704 - 46s - 65ms/step - accuracy: 0.8100 - loss: 0.7516 - val_accuracy: 0.8316 - val_loss: 0.6848 - learning_rate: 5.0000e-04  
Epoch 31/300  
704/704 - 45s - 64ms/step - accuracy: 0.8126 - loss: 0.7440 - val_accuracy: 0.8318 - val_loss: 0.6943 - learning_rate: 5.0000e-04  
Epoch 32/300  
704/704 - 45s - 64ms/step - accuracy: 0.8112 - loss: 0.7460 - val_accuracy: 0.8432 - val_loss: 0.6669 - learning_rate: 5.0000e-04  
Epoch 33/300  
704/704 - 45s - 64ms/step - accuracy: 0.8174 - loss: 0.7314 - val_accuracy: 0.8398 - val_loss: 0.6647 - learning_rate: 5.0000e-04  
Epoch 34/300  
704/704 - 44s - 63ms/step - accuracy: 0.8183 - loss: 0.7348 - val_accuracy: 0.8440 - val_loss: 0.6704 - learning_rate: 5.0000e-04  
Epoch 35/300  
704/704 - 45s - 65ms/step - accuracy: 0.8182 - loss: 0.7373 - val_accuracy: 0.8342 - val_loss: 0.6877 - learning_rate: 5.0000e-04  
Epoch 36/300  
704/704 - 44s - 63ms/step - accuracy: 0.8229 - loss: 0.7223 - val_accuracy: 0.8474 - val_loss: 0.6442 - learning_rate: 5.0000e-04  
Epoch 37/300  
704/704 - 45s - 64ms/step - accuracy: 0.8237 - loss: 0.7163 - val_accuracy: 0.8466 - val_loss: 0.6600 - learning_rate: 5.0000e-04  
Epoch 38/300  
704/704 - 45s - 64ms/step - accuracy: 0.8276 - loss: 0.7147 - val_accuracy: 0.8490 - val_loss: 0.6590 - learning_rate: 5.0000e-04  
Epoch 39/300  
704/704 - 45s - 64ms/step - accuracy: 0.8247 - loss: 0.7154 - val_accuracy: 0.8600 - val_loss: 0.6511 - learning_rate: 5.0000e-04  
Epoch 40/300  
704/704 - 44s - 63ms/step - accuracy: 0.8240 - loss: 0.7185 - val_accuracy: 0.8538 - val_loss: 0.6412 - learning_rate: 5.0000e-04  
Epoch 41/300  
704/704 - 44s - 63ms/step - accuracy: 0.8200 - loss: 0.7088 - val_accuracy: 0.8566 - val_loss: 0.6317 - learning_rate: 5.0000e-04
```



```
313/313 - 30ms/step - accuracy: 0.9113 - loss: 0.4154  
Test Accuracy: 0.9103000164031982  
Test Loss: 0.4117858409021592  
1 # Fetch the raw image from GitHub  
2 url = "https://raw.githubusercontent.com/FarzadNekouee/Keras-CIFAR10-CNN-Model/master/truck_sample.png"  
3 resp = urllib.request.urlopen(url)  
4 image = np.asarray(bytearray(resp.read()), dtype="uint8")  
5 image = cv2.imdecode(image, cv2.IMREAD_UNCHANGED)  
6  
7 # Convert the image from BGR to RGB  
8 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```



The code:

```
import warnings
warnings.filterwarnings('ignore')

import cv2
import numpy as np
import urllib.request
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from keras.datasets import cifar10
from tensorflow.keras.preprocessing.image import ImageDataGenerator # Corrected import
from keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Conv2D, MaxPooling2D
from keras.layers import Dropout, Flatten, BatchNormalization
from keras.regularizers import l2
from keras.optimizers import Adam
from keras.callbacks import ReduceLROnPlateau, EarlyStopping
from keras.models import load_model

X_train, X_valid, y_train, y_valid = train_test_split(X_train, y_train, test_size=0.1, random_state=0)

print('Train Images Shape: ', X_train.shape)
print('Train Labels Shape: ', y_train.shape)
```



```
print('\nValidation Images Shape: ', X_valid.shape)
print('Validation Labels Shape: ', y_valid.shape)

print('\nTest Images Shape:   ', X_test.shape)
print('Test Labels Shape:    ', y_test.shape)

# CIFAR-10 classes
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

# Create a new figure
plt.figure(figsize=(15,15))

# Loop over the first 25 images
for i in range(64):
    # Create a subplot for each image
    plt.subplot(8, 8, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

    # Display the image
    plt.imshow(X_train[i])

    # Set the label as the title
    plt.title(class_names[y_train[i][0]], fontsize=12)

# Display the figure
plt.show()

# Convert pixel values data type to float32
X_train = X_train.astype('float32')
X_test  = X_test.astype('float32')
X_valid = X_valid.astype('float32')

# Calculate the mean and standard deviation of the training images
mean = np.mean(X_train)
std  = np.std(X_train)

# Normalize the data
```

```
# The tiny value 1e-7 is added to prevent division by zero
X_train = (X_train-mean)/(std+1e-7)
X_test = (X_test-mean)/(std+1e-7)
X_valid = (X_valid-mean)/(std+1e-7)

y_train = to_categorical(y_train, 10)
y_valid = to_categorical(y_valid, 10)
y_test = to_categorical(y_test, 10)

# Data augmentation
data_generator = ImageDataGenerator(
    # Rotate images randomly by up to 15 degrees
    rotation_range=15,

    # Shift images horizontally by up to 12% of their width
    width_shift_range=0.12,

    # Shift images vertically by up to 12% of their height
    height_shift_range=0.12,

    # Randomly flip images horizontally
    horizontal_flip=True,

    # Zoom images in by up to 10%
    zoom_range=0.1,

    # Change brightness by up to 10%
    brightness_range=[0.9,1.1],

    # Shear intensity (shear angle in counter-clockwise direction in degrees)
    shear_range=10,

    # Channel shift intensity
    channel_shift_range=0.1,
)

# Initialize a sequential model
model = Sequential()

# Set the weight decay value for L2 regularization
```

weight_decay = 0.0001

```
# Add the first convolutional layer with 32 filters of size 3x3
model.add(Conv2D(filters=32, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay),
input_shape=X_train.shape[1:]))
```

```
# Add batch normalization layer
model.add(BatchNormalization())
```

```
# Add the second convolutional layer similar to the first
model.add(Conv2D(filters=32, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
model.add(BatchNormalization())
```

```
# Add the first max pooling layer with pool size of 2x2
```

```
model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
# Add dropout layer with 0.2 dropout rate
```

```
model.add(Dropout(rate=0.2))
```

```
# Add the third and fourth convolutional layers with 64 filters
```

```
model.add(Conv2D(filters=64, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
```

```
model.add(BatchNormalization())
```

```
model.add(Conv2D(filters=64, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
```

```
model.add(BatchNormalization())
```

```
# Add the second max pooling layer and increase dropout rate to 0.3
```

```
model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
model.add(Dropout(rate=0.3))
```

```
# Add the fifth and sixth convolutional layers with 128 filters
```

```
model.add(Conv2D(filters=128, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
```

```
model.add(BatchNormalization())
```

```
model.add(Conv2D(filters=128, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
```

```
model.add(BatchNormalization())
```

```
# Add the third max pooling layer and increase dropout rate to 0.4
```



```
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(rate=0.4))

# Add the seventh and eighth convolutional layers with 256 filters
model.add(Conv2D(filters=256, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
model.add(BatchNormalization())
model.add(Conv2D(filters=256, kernel_size=(3,3), padding='same', activation='relu',
kernel_regularizer=l2(weight_decay)))
model.add(BatchNormalization())

# Add the fourth max pooling layer and increase dropout rate to 0.5
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(rate=0.5))

# Flatten the tensor output from the previous layer
model.add(Flatten())

# Add a fully connected layer with softmax activation function for outputting class probabilities
model.add(Dense(10, activation='softmax'))

# Set the batch size for the training
batch_size = 64

# Set the maximum number of epochs for the training
epochs = 300

# Define the optimizer (Adam)
optimizer = Adam(learning_rate=0.0005)

# Compile the model with the defined optimizer, loss function, and metrics
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

# Add ReduceLROnPlateau callback
# Here, the learning rate will be reduced by half (factor=0.5) if no improvement in validation loss is
observed for 10 epochs
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=10, min_lr=0.00001)

# Add EarlyStopping callback
# Here, training will be stopped if no improvement in validation loss is observed for 40 epochs.
```

```
# The `restore_best_weights` parameter ensures that the model weights are reset to the values from the
epoch
# with the best value of the monitored quantity (in this case, 'val_loss').
early_stopping = EarlyStopping(monitor='val_loss', patience=40, restore_best_weights=True, verbose=1)

# Fit the model on the training data, using the defined batch size and number of epochs
# The validation data is used to evaluate the model's performance during training
# The callbacks implemented are learning rate reduction when a plateau is reached in validation loss and
# stopping training early if no improvement is observed
model.fit(data_generator.flow(X_train, y_train, batch_size=batch_size),
          epochs=epochs,
          validation_data=(X_valid, y_valid),
          callbacks=[reduce_lr, early_stopping],
          verbose=2)

plt.figure(figsize=(15,6))

# Plotting the training and validation loss
plt.subplot(1, 2, 1)
plt.plot(model.history.history['loss'], label='Train Loss', color='#8502d1')
plt.plot(model.history.history['val_loss'], label='Validation Loss', color='darkorange')
plt.legend()
plt.title('Loss Evolution')

# Plotting the training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(model.history.history['accuracy'], label='Train Accuracy', color='#8502d1')
plt.plot(model.history.history['val_accuracy'], label='Validation Accuracy', color='darkorange')
plt.legend()
plt.title('Accuracy Evolution')

plt.show()

# Use the model to make predictions, evaluate on test data
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=1)

print('\nTest Accuracy:', test_acc)
print('Test Loss: ', test_loss)

# Fetch the raw image from GitHub
```

```
url = "https://raw.githubusercontent.com/FarzadNekouee/Keras-CIFAR10-CNN-Model/master/truck_sample.png"
resp = urllib.request.urlopen(url)
image = np.asarray(bytearray(resp.read()), dtype="uint8")
image = cv2.imdecode(image, cv2.IMREAD_UNCHANGED)

# Convert the image from BGR to RGB
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image)
plt.xticks([])
plt.yticks([])
plt.grid(False)
plt.show()

# Resize it to 32x32 pixels
image = cv2.resize(image, (32,32))

# Normalize the image
image = (image-mean)/(std+1e-7)

# Add an extra dimension because the model expects a batch of images
image = image.reshape((1, 32, 32, 3))

prediction = model.predict(image)

predicted_class = prediction.argmax()

print('Predicted class: ', class_names[predicted_class])
```